REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Fabián San Martín

8 de abril de 2026

16:25

1. Entorno experimental

A continuacion se describe el entorno experimental utilizado para llevar a cabo los experimentos, incluyendo detalles sobre el hardware, software y las condiciones de entrada empleadas.

- **Hardware utilizado:** Intel i5-11400h, 16 GB RAM DDR4 3200Mt/s, almacenamiento SSD NVME 1Tb GB.
- **Software utilizado:** Sistema operativo Linux Mint 22.3 (instalación nativa, no subsistema ni VM), compilador g++ 13.3.0 (con optimización -O2), Python 3.13, librerías relevantes: Chrono (C++), Pandas, NumPy, MatPlotLib.

2. Resultados y Análisis

Gráficos obtenidos. Los resultados se guardan como `<algoritmo>.csv` en la ruta `code/<problema>/data/measurements`

Cada registro incluye tamaño de entrada, dimensión y tiempo de ejecución en ms. Para generar los datos, entrar a `code/<problema>/`, ejecutar `make` y luego `./<problema>`. Los CSV quedarán en data/-measurements.

Para generar las gráficas, entrar a `code/<problema>/scripts/` y ejecutar el comando:

```
python3 plot_generator.py
```

Se requiere módulos de Python3: Pandas, NumPy, Matplotlib, Seaborn y Warnings.

Algoritmos de Ordenamiento

Análisis de Tiempo

Aquí se presentan los resultados obtenidos. La Figura 1 compara todos los algoritmos en relación con Tiempo ms y Tamaño de la entrada definidas por el enunciado del problema. Usamos la escala logarítmica para facilitar la visualización de los datos.

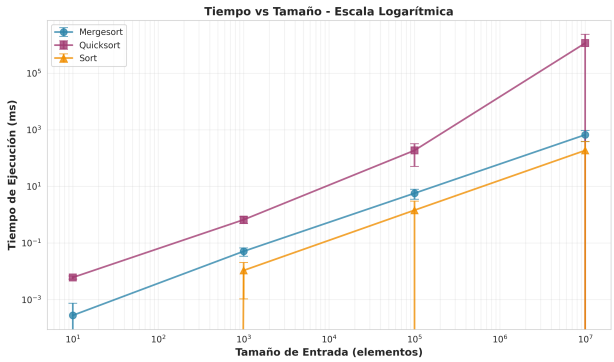

Figura 1: Todos los algoritmos Sorts.

El tiempo registrado de Sort para datos de tamaño menor a 10^3 *fue de 0 ms por su rapidez.*

En la **Figura 1** podemos observar nuestro segundo peor algoritmo estudiado, comenzaremos analizaremos Quick Sort, con la implementación de Random Pivot, que en teoría debería ser el mejor algoritmo gracias a su media de tiempo de $O(n * log(n))$, pero al entrar en su peor caso, que es $O(n^2)$, y al encontrar matrices de tamaño 10^7, y gracias a las repetidas llamadas recursivas se se puede encontrar con el problema de Segmentation Fault, que significa un rebalse de la memoria Stack, por lo que puede tener problemas ejecutando un ordenamiento de tamaño 10^7.

Después, podemos observar el Merge Sort, como lo visto en clases, este algoritmo tiene un tiempo de ejecución de $O(n * log(n))$, y es el segundo mejor algoritmo después de Sort de C++. Y claramente podemos observar su comportamiento logarítmico en la gráfica. No es el mejor, ya que en arreglos grandes, Merge Sort utilizará memoria adicional para combinar las sublistas, a diferencia de Quick Sort y Sort que son in-place.

Por último, tenemos Sort, que en la práctica es el mejor algoritmo de todos, este algoritmo está basado en Introsort, que es una combinación de 3 métodos: Quick Sort, Heap Sort y Insertion Sort. El algoritmo comienza utilizando Quick Sort y cuando detecta que la recursión es profunda cambia automáticamente a Heap Sort para evitar el peor de los casos $O(n^2)$. Por esto mismo es el mejor de todos ya que minimiza, casi anula completamente alguna chance de entrar a su peor caso, manteniendo su tiempo de ejecución en $O(n * log(n))$.

Análisis de Memoria

En el Análisis de Memoria se muestran cuatro métricas: *Delta RSS*, *Peak Memory*, *Delta Peak desde Inicio* y *RSS Después*. Algunas métricas representan memoria total del proceso y otras representan creci-

miento real debido a cómo se implementa el programa.

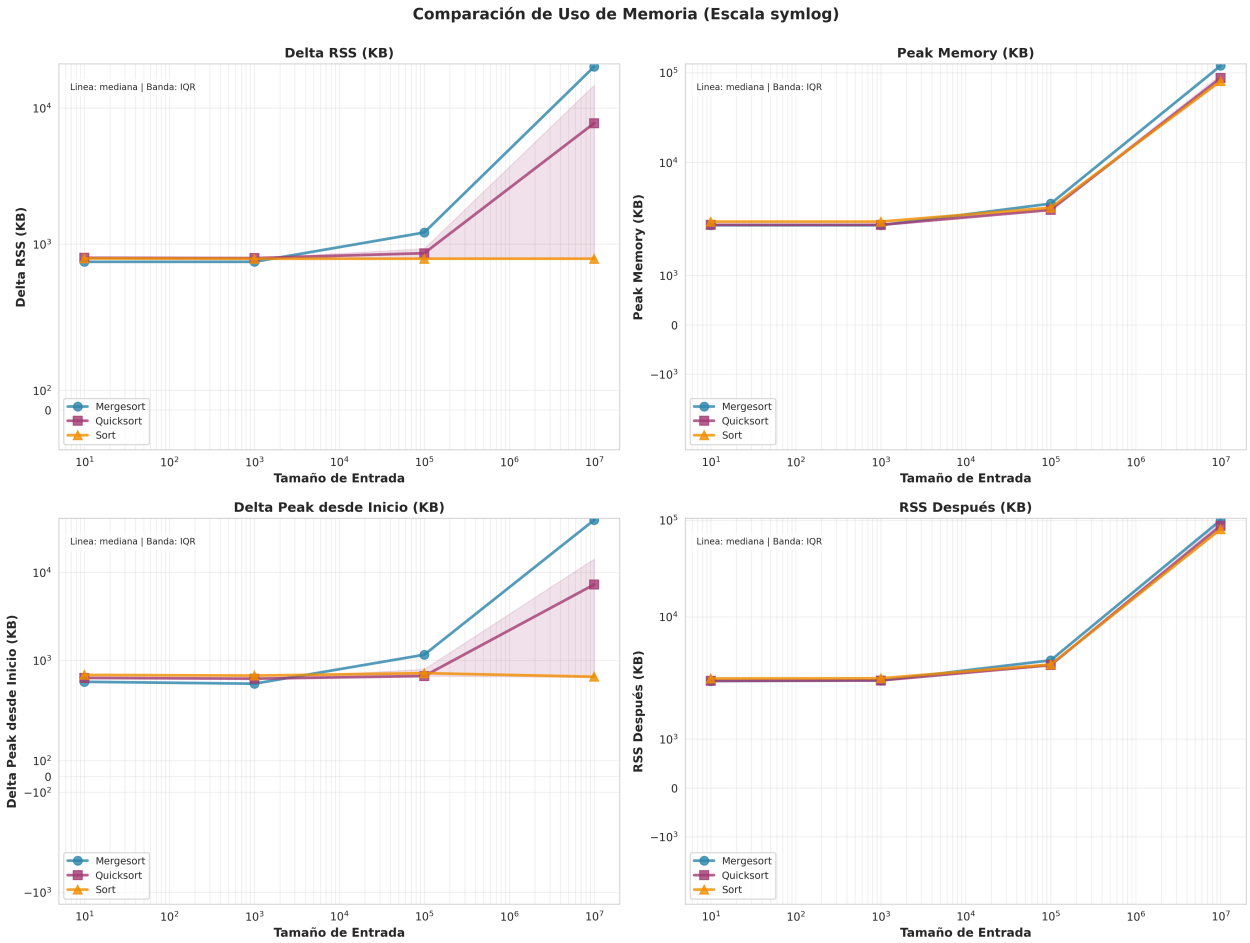

Figura 2: Todos los algoritmos Sorts Memoria vs Tamaño.

Primero, al observar *Peak Memory* y *RSS Después*, los tres algoritmos parecen bastante parecidos en tamaños pequeños y medianos. Entendemos esto porque estas métricas incluyen un costo inicial de importar librerías y funciones para medir su rendimiento. En otras palabras, para entradas pequeñas, el costo fijo domina y oculta diferencias finas entre algoritmos.

Segundo, cerca de 10^5 y 10^7, se pueden observar cambios notables donde los algoritmos empiezan a diferenciarse entre sí. Esto se debe a la forma que asignan la memoria, generando aumentos no lineales.

Ahora, **comparando algoritmo por algoritmo** en (*Delta RSS* y *Delta Peak desde Inicio*), se aprecia mejor el comportamiento real:

- **Merge Sort** tiende a requerir más memoria adicional en tamaños grandes porque necesita arreglos temporales para combinar sublistas. Este comportamiento es esperado por su diseño, ya que no es in-place.

- **Quick Sort** es eficiente en memoria por ser in-place, pero tiene el riesgo de caer en recursión profunda en ciertos casos (por ejemplo, por una mala elección de pivote). Esto puede aumentar el consumo de stack y provocar problemas como *Segmentation Fault* en tamaños muy grandes.

- **Sort** de C++ (Introsort) se mantiene como la alternativa más estable en la práctica: conserva buena eficiencia temporal y su uso de memoria adicional es contenido, gracias a que combina Quick Sort, Heap Sort e Insertion Sort según el caso.

A continuación, se presenta el análisis para los algoritmos de multiplicación de matrices.

Algoritmos de Multiplicación de Matrices

Análisis de Tiempo

Aquí los resultados obtenidos Tiempo (ms) vs Tamaño de la entrada N. Los datos de Tamaño son $\{2^4, 2^6, 2^8, 2^{10}\}$.

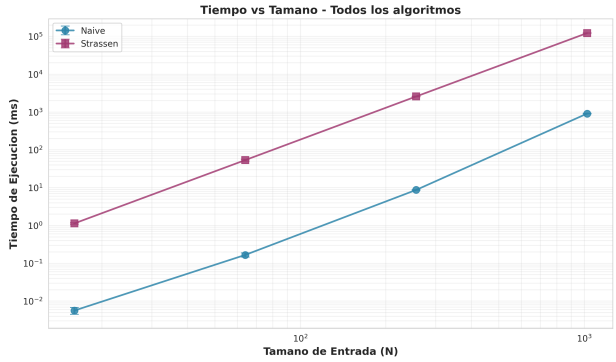

Figura 3: Todos los algoritmos de Multiplicación.

En la **Figura 3** podemos observar los datos obtenidos de los algoritmos Naive y Strassen al intentar multiplicar matrices cuadradas de tamaños $\{2^4, 2^6, 2^8, 2^{10}\}$. Podemos observar que, aunque el algoritmo Naive tiene un tiempo de ejecución de $O(n^3)$ y Strassen tiene una complejidad de $O(n^{log_2 7})$, Naive sigue siendo superior al algoritmo Strassen en todo momento, incluso viendo después del tamaño 2^{10} aún se puede observar en el gráfico que Strassen es peor que Naive. Nos encontramos en una situación bizarra en donde el algoritmo con peor complejidad teórica es mejor en la práctica.

Por otro lado, tenemos al algoritmo **Strassen**, que en teoría debería ser mejor que **Naive**, ya que su tiempo de ejecución es de $O(n^{log_2 7})$. Además, al tener que hacer muchas operaciones con matrices temporales, el consumo de memoria es mucho mayor.

¿Pero por qué sucede esto? Después de analizar lo que estaba sucediendo, llegué a la conclusión que el algoritmo de Strassen puro es ineficiente para matrices pequeñas, y por esto mismo termina siendo peor en todo momento que Naive, ya que utiliza Strassen en todo momento sin importar el tamaño de la matriz. Una mejor implementación de Strassen debería utilizar Naive para matrices pequeñas, y Strassen para matrices grandes, ya que Strassen tiene una sobrecarga mayor por la cantidad de operaciones adicionales que debe realizar, además que se podría repartir la tarea en múltiples hilos del procesador para aprovechar mejor los CPUs modernos. De esta manera se entiende que Strassen es mejor algoritmo cuando las matrices son muy grandes y su implementación es conjunto a Naive.

Análisis de Memoria

En el análisis de memoria para multiplicación de matrices también se consideran cuatro métricas: *Delta RSS*, *Peak Memory*, *Delta Peak desde Inicio* y *RSS Después*.

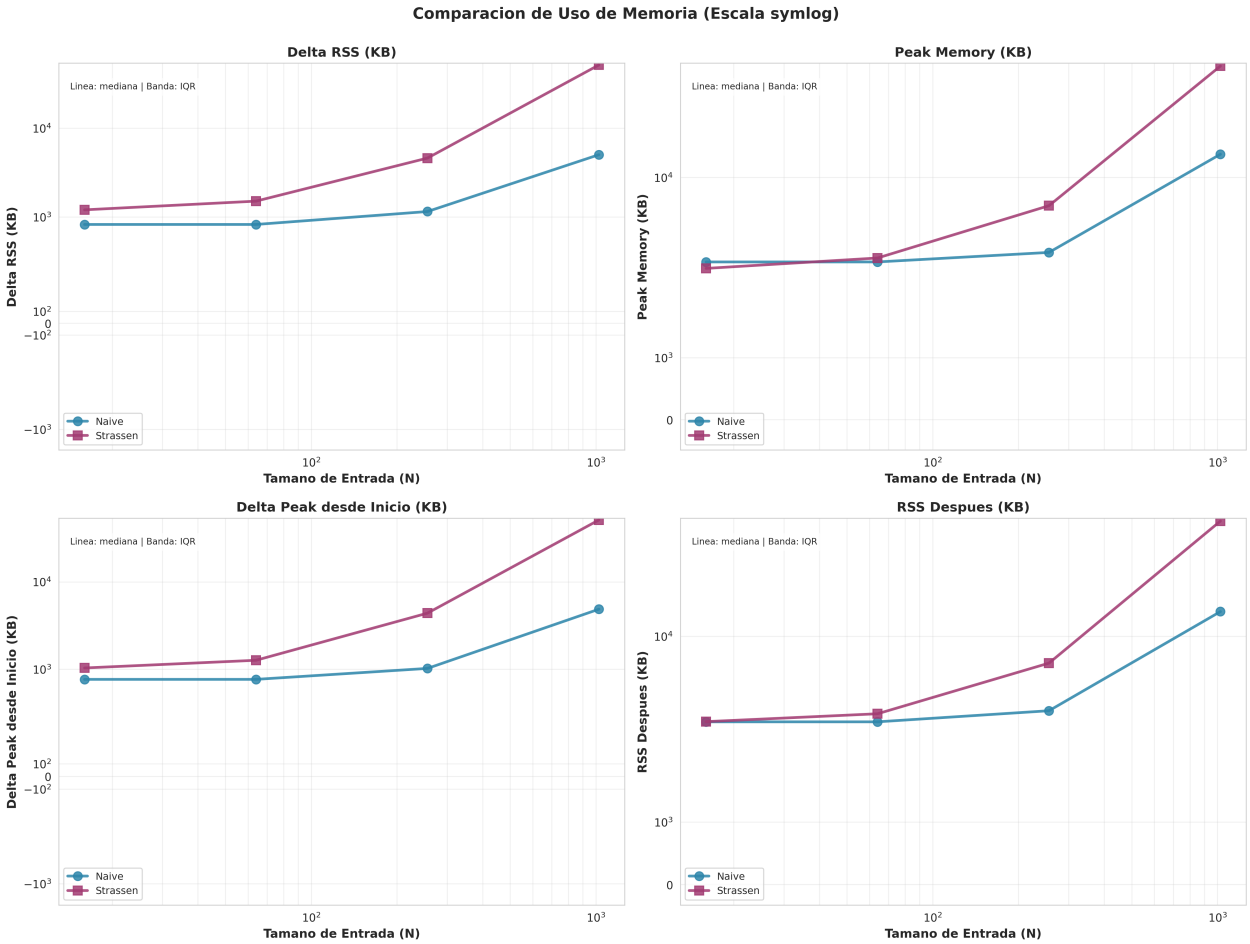

Figura 4: Uso de memoria en multiplicación de matrices (Naive vs Strassen).

Primero, en *Peak Memory* y *RSS Después* se aprecia que ambos algoritmos aumentan su consumo al crecer el tamaño de entrada, con saltos más notorios en los tamaños grandes. Este comportamiento se esperaba porque, además de la memoria de datos, el proceso arrastra costos de runtime, allocator y reservas de memoria.

Segundo, al observar las métricas (*Delta RSS* y *Delta Peak desde Inicio*), se nos muestra muy claramente el problema: Strassen requiere más memoria adicional que Naive, esto se nota especialmente para tamaños grandes. Esto coincide con la teoría y con la implementación práctica, ya que Strassen recursivamente crea múltiples submatrices temporales y realiza más operaciones intermedias durante la recursión.

Naive, en cambio, muestra un patrón más estable en la alocación de memoria. Aunque su complejidad temporal sea $O(n^3)$, su estructura de cálculo es más directa sin cálculos redundantes y con menos sobrecarga de estructuras auxiliares en comparación de Strassen que usa Dividir y Conquistar.

Se observa de Strassen una de las maneras más comunes para compensar la rapidez: el aumento del uso de la memoria con la complejidad temporal ($O(n^{\log_2 7})$). En este caso, como el algoritmo no implementa Strassen de forma híbrida, su costo de memoria termina empeorando su rapidez.

En conclusión, se entiende que sin una implementación inteligente de Strassen, será como un tiro de la culata al final de cuentas, para estos tamaños y esta implementación, Naive simplemente es mejor en práctica en tiempo y memoria. Strassen mantiene su potencial para escalas matrices mayores cuando se implementa de manera híbrida (Strassen y Naive combinados).

3. Conclusiones

Los experimentos realizados evidencian que la eficiencia teórica de un algoritmo no siempre se traduce en mejor desempeño práctico sin una aplicación inteligente. El caso de Strassen muestra que una mala implementación puede hacer que un algoritmo supuestamente teóricamente superior rinda peor que uno más simple como Naive. Por otro lado, la robustez y optimización de algoritmos populares, como Sort de C++, su uso suelen ser las más confiables para la mayoría de los casos.

Sintetizando todo, los resultados obtenidos permiten concluir que la elección de algoritmos debe considerar tanto al implementación como el contexto del problema. Realizar estos experimentos de primera mano resultó una ser una muy buena experiencia para mi criterio de uso de algoritmos.